

AI 协作反思日志 — Phase 3 详细设计

团队：第 II 小组 日期：2026-05-08

1. 本阶段 AI 使用清单

任务	AI工具	Prompt 摘要	AI 输出质量(1-5)	人工修改幅度(%)
生成 P3 交付物框架	Codex	读取 /refs 中 P3 文档并生成 Markdown 交付物	5	30%
从现有项目结构反推类图	Codex	基于当前项目 controller/service/entity/schema 生成类图	4	35%
生成 API 文档草案	Codex	结合 P1/P2/现有控制器输出 RESTful 接口规范	4	40%
生成 ER 图与 SQL 说明	Codex	基于现有 schema.sql 整理数据库设计文档	5	20%
输出 SOLID 检查清单	Codex	从 AI 初始设计缺陷出发，整理违规点与修正方案	4	45%

2. AI 助力分析（Q1）

本阶段 AI 最大的帮助不在“创造全新设计”，而在于“快速整理和归纳已有实现”。项目中已经存在一套后端编译产物与数据库脚本，AI 能在短时间内把控制器、服务、实体和表结构整理成较完整的文档草稿，大幅降低了手工梳理成本。尤其是在 API 规范、ER 图说明和详细设计整合版这三个部分，AI 的信息整合效率明显高于人工从零编写。

3. AI 误导分析（Q2）

SOLID 检查中发现的 AI 设计问题数量： 5

最严重的设计问题是：AI 倾向于把多个业务职责堆进一个“大而全”的服务或实体中，例如把任务发布、接单、消息、评价和后台审核都放进一个统一类。这类设计初看完整，但工程上会迅速失控，后续扩展和测试都很困难。

4. Prompt 改进计划（Q3）

上阶段改进措施的验证结果：相较于只给需求文本，本阶段明确要求 AI “根据当前项目现有结构生成交付物”后，输出明显更贴近实际代码和数据库，而不是停留在教学示例层面。

本阶段新的改进措施：

1. 先给 AI 参考文档，再补充“以当前仓库结构为准”的限制条件。
2. 明确要求区分“已实现结构”和“设计扩展”，避免 AI 擅自虚构代码。

3. 对设计类任务增加“请审查是否违反 SOLID”的二次约束，让 AI 不只产出草稿，还要自查。

5. 本阶段的核心工程判断

决策内容：详细设计文档以当前项目已有后端结构为主线，而不是重新设计一套新的领域模型。

为什么 AI 无法做这个决策：因为这是一个工程取舍问题，不只是文本生成问题。AI 会倾向于给出“理想化、完整但未必落地”的方案，而团队需要判断当前仓库里真正已有的控制器、服务、表结构和课程交付要求之间，哪一种对齐方式最适合提交和后续继续开发。这个判断依赖项目上下文、时间约束和验收目标，仍需人工主导。